

Artificial Intelligence

Course Code: CS-202

Topic: Search Algorithms – Informed BFS, A*

Instructor: Habiba Arshad

Week-3

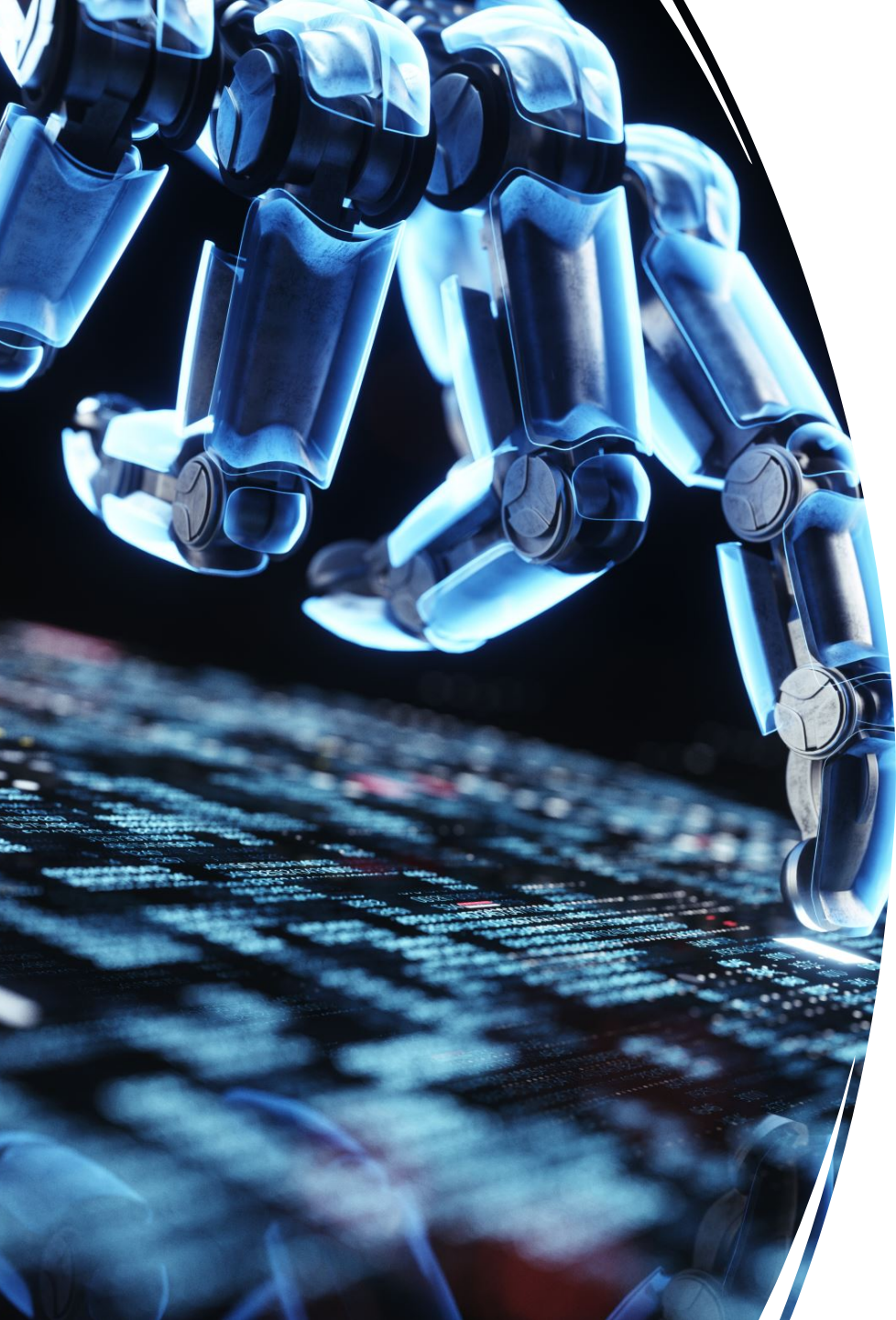
This week lecture will cover the following CLO:

- **CLO 2:** . Implement artificial intelligence techniques and case studies



Search

- The process of Finding a path to desired goal state from all possible future states.
- The Solution to a search problem is a sequence of actions, called the plan that transforms the start state to the goal state.
- This plan is achieved through search algorithms.



Kinds of Search

- There are 2 kinds of search based on whether they use information about the goal.
- **Uninformed search algorithms**-algorithms have no additional information on the goal node other than the one provided in the problem definition. The plans to reach the goal state from the start state differ only by the order and/or length of actions. Uninformed search is also called Blind search.
- **Informed search algorithms**, the algorithms have information on the goal state, which helps in more efficient searching. This information is obtained by something called a heuristic.

What is Heuristic?

- A **heuristic function** is a rule or formula that estimates the cost or distance from a given state to the goal.
- It helps informed search algorithms make better decisions by guiding them toward an optimal solution efficiently.



Search Algorithm

```
graph TD; A[Search Algorithm] --> B[Uniformed/Blind]; A --> C[Informed Search]; B --> D[Breadth first search]; B --> E[Uniform cost search]; B --> F[Depth first search]; B --> G[Depth limited search]; B --> H[Iterative deeping depth first search]; B --> I[Bidirectional search]; C --> J[Best First Search]; C --> K[A*search];
```

Uniformed/Blind

Breadth first search

Uniform cost search

Depth first search

Depth limited search

Iterative deeping depth
first search

Bidirectional search

Informed Search

Best First Search

A*search

Informed (Heuristics) Search Algorithms

- Best-First Search (BFS)
- **Best-First Search (BFS)** is a **heuristic-based search algorithm** that selects the most promising node based on a given **evaluation function**. It uses a **priority queue** to explore paths that appear closer to the goal first.



Example of Best-First Search

- Consider a graph where the heuristic function $h(n)$ represents the straight-line distance to the goal. BFS always expands the closest-looking node first.
- Use Case: Pathfinding in a city
- Nodes = Cities
- Edges = Roads between cities
- Heuristic $h(n)$ = Straight-line distance or Euclidean distance to the destination
- The algorithm expands the city that appears closest to the goal.

Steps of Best-First Search

Initialize

- Create two lists:
 - **OPEN (priority queue)**: Stores nodes to be explored, sorted by $h(n)$.
 - **CLOSED (visited list)**: Keeps track of explored nodes to prevent re-exploration.
- Add the **start node** to the OPEN list.

Search Process

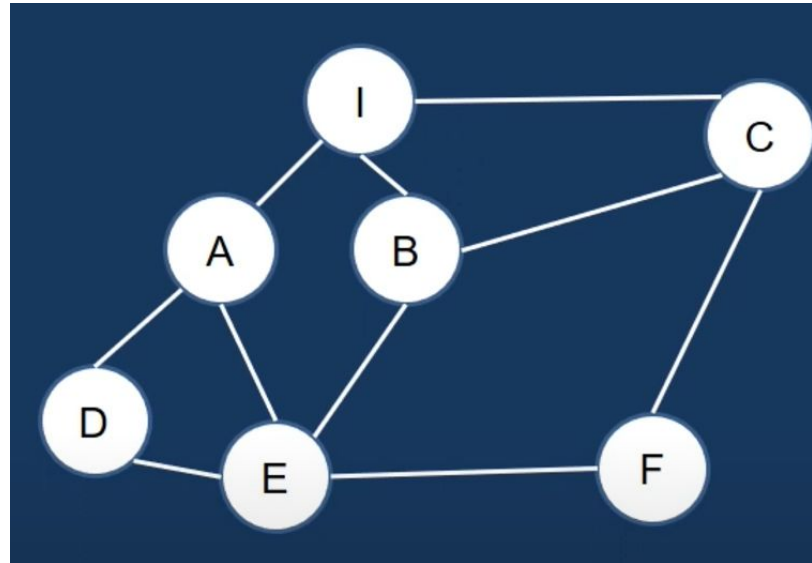
- While the OPEN list is **not empty**:
 - Select and remove the node **N** with the **lowest $h(n)$** from OPEN.
 - Move **N** to the CLOSED list and store its parent information.
 - If **N is the goal**, exit and reconstruct the path by backtracking from the goal to the start.
 - Otherwise, expand **N** to generate its successors.
 - Add all successors to the OPEN list (**if not already explored in CLOSED**).
 - Reorder the OPEN list based on $h(n)$ so that the most promising node is expanded next.

End Condition

- If the goal is reached, return the path.
- If the OPEN list becomes empty without finding the goal, return **failure**.

Example

Consider the given graph to solve a problem statement where we had to find out the shortest path from city I to F with the help of best first search algorithm and best first algorithm uses distance cost called as heuristic values.



$$I - F = 45$$

$$A - F = 53$$

$$B - F = 40$$

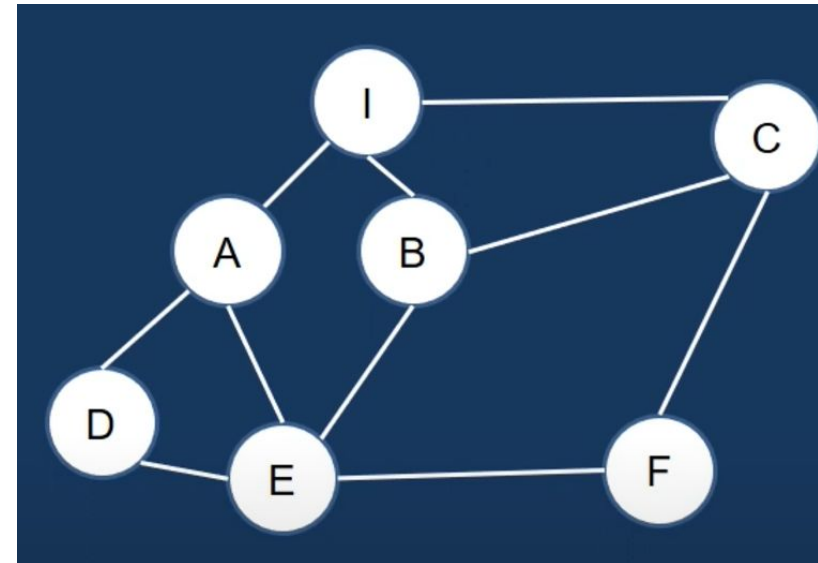
$$C - F = 42$$

$$D - F = 57$$

$$E - F = 44$$

Example Solution

Open (Unvisited)	I:45
Close (Visited)	-
Open (Unvisited)	B:40 , C: 42, A:53
Close (Visited)	I
Open (Unvisited)	C:42, E:44, A:53
Close (Visited)	I,B
Open (Unvisited)	F:0, E:44, A:53
Close (Visited)	I,B,C
Open (Unvisited)	E:44, A:53
Close (Visited)	I,B,C,F



I -> B -> C -> F

$$I - F = 45$$

$$A - F = 53$$

$$B - F = 40$$

$$C - F = 42$$

$$D - F = 57$$

$$E - F = 44$$

Informed (Heuristics) Search Algorithms

A* Search Algorithm

- A* search algorithm finds the shortest path through the search like heuristic function.
- It uses $h(n)$ & Cost to reach the node n from the start stage $g(n)$.
- This is an admissible algorithm that ensures that it must give optimal solution
- A* use search heuristic as well as the cost to reach the node hence we combine both cost as
 - **$f(n) = g(n) + h(n)$**
 - $g(n)$ is the actual cost from the start node to node n .
 - $h(n)$ is the heuristic estimate of the cost from node n to the goal.
 - $f(n)$ represents the total estimated cost of the path through node n .

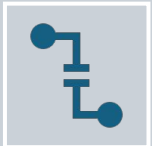
Features



Optimal: It always finds the least-cost path if $h(n)$ is admissible (never overestimates).



Complete: Will always find a solution if one exists.



Efficient: Explores fewer nodes compared to other uninformed algorithms.

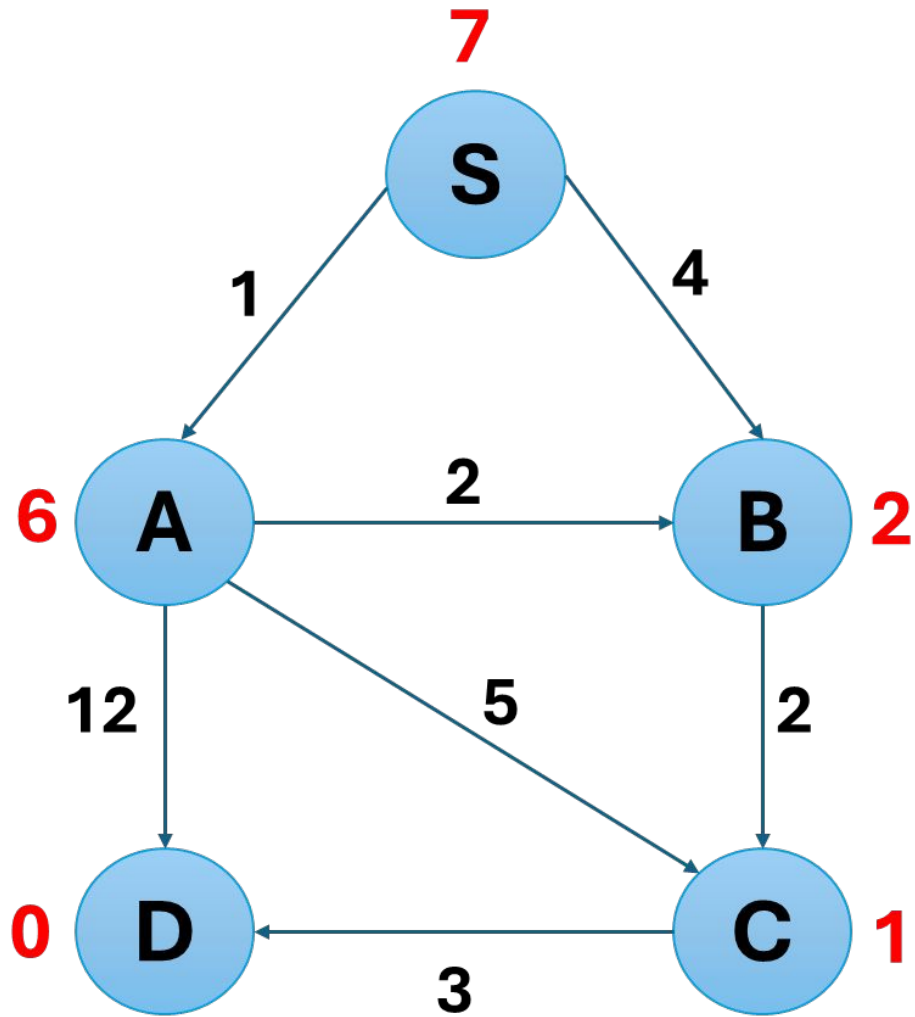
Steps

Initialize:

- Create an **OPEN list** (priority queue): nodes to be explored.
- Create a **CLOSED list**: nodes already explored.
- Add the **start node** to OPEN.

Loop:

- While OPEN is not empty:
 1. Pick the node **n** with the **lowest $f(n)$** from OPEN.
 2. If **n** is the goal node → **Backtrack** to construct the path → **Finish**.
 3. Move **n** from OPEN to CLOSED.
 4. Expand node **n** (generate successors).
 5. For each successor:
 - Calculate **$g(\text{succ})$** , **$h(\text{succ})$** , and **$f(\text{succ})$** .
 - If it's already in OPEN or CLOSED with a lower **f**, **ignore** it.
 - Otherwise, add it to OPEN and record **n** as its parent.
- If OPEN is empty and the goal hasn't been found → **Failure**.



A* Search Algorithm

$$f(n) = g(n) + h(n)$$

- $g(n)$ is the actual cost from the start node to node n .
- $h(n)$ is the heuristic estimate of the cost from node n to the goal.
- $f(n)$ represents the total estimated cost of the path through node n .

Informed (Heuristics) Search Algorithms

A* Search Algorithm

Start State : A
Goal State: F

